Visualización de características en redes convolucionales entrenadas sobre Pl@ntNet-300K

Informe técnico

David Fernández Fernández y Javier Santamaría Pascual

Noviembre de 2026

Resumen

Este informe describe la aplicación de las técnicas de visualización de características de Olah, Mordvintsev y Schubert (2017) a redes ResNet entrenadas para identificar 1081 especies de plantas en el conjunto Pl@ntNet-300K. La visualización por optimización busca la imagen de entrada que maximiza la respuesta de una unidad interna de la red, manteniendo fijos los pesos del modelo. Se describen los objetivos de optimización (neurona, canal, capa, clase y combinaciones de unidades), la parametrización de la imagen en el dominio de la frecuencia, la robustez a transformaciones y las penalizaciones que evitan el ruido adversarial, así como dos extensiones: la interpolación entre neuronas y la generación de ejemplos diversos de una misma unidad. Sobre estas técnicas se ha construido una herramienta de inspección que, a partir de una fotografía, identifica la especie, ordena las neuronas que se activan, localiza en qué región de la imagen lo hacen y genera el patrón que cada una detecta. La ablación de la robustez reproduce el argumento central del artículo: sin ella, la activación alcanzada aumenta un 45 % mientras la aspereza de la imagen crece un 49 %.

1. Introducción

Una red neuronal entrenada resuelve su tarea, pero no explica cómo lo hace. Sus parámetros no se pueden leer directamente, y saber qué ha aprendido cada una de sus unidades internas es un problema abierto de la interpretabilidad. Una forma de abordarlo es mirar qué fotografías del conjunto de datos activan más una unidad. Otra, la que se estudia en este trabajo, es construir esas fotografías: partiendo de ruido y modificando la imagen de entrada, mediante descenso por gradiente, hasta que la unidad elegida responda todo lo posible. La imagen resultante muestra el patrón que esa unidad detecta.

El artículo de referencia, *Feature Visualization* [1], reúne y sistematiza las técnicas necesarias para que este procedimiento produzca imágenes interpretables. Su resultado es cualitativo, en forma de figuras, y no establece ninguna métrica que reproducir. Este trabajo aplica esas técnicas a otro dominio, el de la identificación de especies vegetales, sobre redes ResNet entrenadas en Pl@ntNet-300K [2], y construye sobre ellas una herramienta de inspección.

El informe se organiza como sigue. La sección 2 formula el problema de optimización. La sección 3 describe el modelo y los datos. La sección 4 detalla cada una de las técnicas implementadas, y la sección 5, la herramienta construida sobre ellas. Las secciones 6 y 7 definen las métricas y la configuración experimental, y la sección 8 presenta los resultados. El informe termina con las adaptaciones respecto al artículo y las conclusiones.

2. Fundamentos

2.1. Activaciones de una red convolucional

Al procesar una imagen $\mathbf{x}$, cada capa convolucional n de la red produce un tensor de activaciones $A^{(n)}(\mathbf{x}) \in \mathbb{R}^{C \times H \times W}$. Cada uno de sus C *canales* es la respuesta de un filtro aprendido, y cada

canal es a su vez una rejilla de $H \times W$ posiciones, cada una de las cuales corresponde a una región de la imagen de entrada, su campo receptivo. Llamamos *neurona* a un elemento concreto $A_{z,i,j}$ del tensor: el canal z en la posición (i, j). La última capa de la red no es convolucional: produce un vector $\boldsymbol{\ell} \in \mathbb{R}^K$ de *logits*, una puntuación por clase, que la función softmax convierte en probabilidades.

2.2. Visualización por optimización

Sea h una función que resume en un escalar la parte de la red que se quiere estudiar, por ejemplo la activación media de un canal. La visualización por optimización resuelve

$$\mathbf{x}^* = \arg\max_{\mathbf{x}} \; h\big(A^{(n)}(\mathbf{x})\big) - \sum_r \lambda_r R_r(\mathbf{x}), \tag{1}$$

donde los términos R_r son penalizaciones sobre la imagen, con pesos $\lambda_r \geq 0$. El problema se resuelve por ascenso de gradiente sobre $\mathbf{x}$. Los pesos de la red permanecen congelados durante todo el proceso: el gradiente se propaga a través del modelo hasta la entrada, y la única variable que se modifica es la imagen. Esta es la diferencia esencial con el entrenamiento, en el que la entrada es fija y lo que cambia son los pesos.

2.3. El problema del ruido adversarial

Resuelto de forma directa sobre los píxeles, el problema (1) no produce imágenes interpretables. El optimizador encuentra patrones de alta frecuencia, casi imperceptibles, que activan la unidad con gran intensidad sin parecerse a ningún objeto: es el mismo fenómeno que permite construir ejemplos adversariales. La red responde con fuerza a estímulos que no aparecen en imágenes naturales, y el optimizador los encuentra antes que los patrones reales. Buena parte de las técnicas descritas en la sección 4 existen para impedirlo. Actúan de tres maneras: cambiando el espacio en el que se optimiza la imagen, exigiendo que el patrón resista transformaciones y penalizando explícitamente las altas frecuencias.

3. Modelo y datos

3.1. Pl@ntNet-300K

Pl@ntNet-300K [2] contiene 306 146 fotografías de 1081 especies vegetales, tomadas por usuarios de la aplicación Pl@ntNet, con una partición oficial en entrenamiento, validación y prueba. La distribución de clases es de cola larga: la mediana es de 21 imágenes de entrenamiento por especie. Este trabajo no entrena ningún modelo. Usa los pesos publicados por los autores del conjunto, y del conjunto de datos solo necesita los metadatos, para traducir clases a nombres de especie, y un número reducido de fotografías del conjunto de prueba.

3.2. Arquitectura

Los autores de Pl@ntNet-300K publican modelos ResNet [3] preentrenados de 18, 50 y 152 capas. Todos los resultados de este informe corresponden al ResNet-18, de 11,7 millones de parámetros; las técnicas no dependen de la arquitectura y se aplican igual a las otras dos. La figura 1 muestra la estructura del ResNet-18: una etapa inicial que reduce la resolución, cuatro etapas de bloques residuales que la reducen a la mitad mientras duplican el número de canales, y una capa lineal final con una salida por especie. Cada bloque residual aplica dos convoluciones y suma el resultado a su propia entrada a través de una conexión residual, o *skip connection*. Esa suma permite que

el gradiente atraviese la red sin degradarse, lo que hace posible entrenar redes profundas; cuando el bloque reduce la resolución o cambia el número de canales, la conexión incluye una convolución 1×1 que ajusta el tamaño.

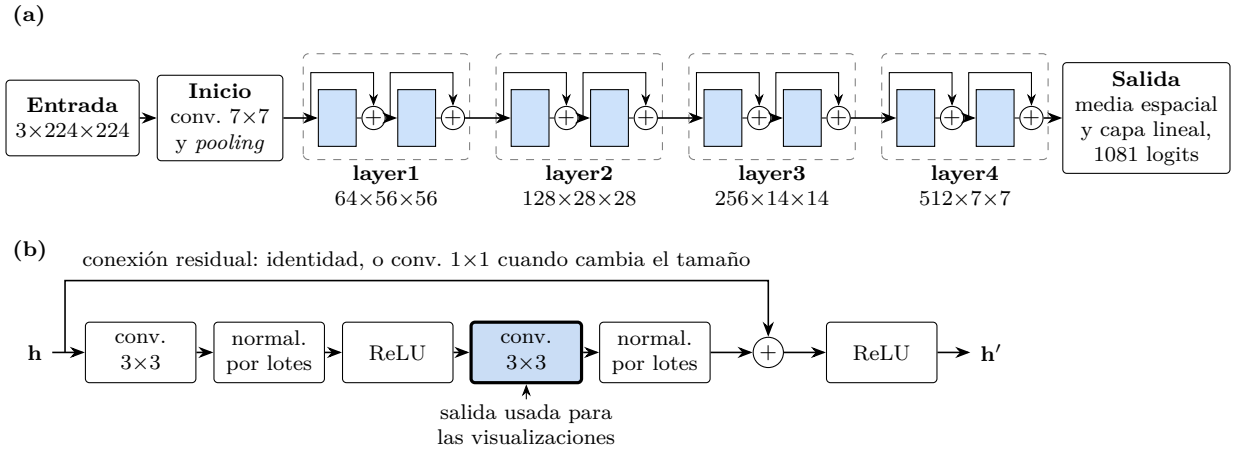


Figura 1: Arquitectura del ResNet-18 para una entrada de 224×224 píxeles. (a) Vista general: tras una etapa inicial que reduce la resolución, cuatro etapas de dos bloques residuales cada una; las cifras indican canales $\times$ alto $\times$ ancho de la salida de cada etapa, y las líneas superiores, las conexiones residuales. (b) Detalle de un bloque residual. Las visualizaciones se calculan sobre la segunda convolución del último bloque de cada etapa, resaltada.

Las capas sobre las que se visualiza son la segunda convolución del último bloque de cada etapa (figura 1b), *antes* de la normalización, de la suma residual y de la función ReLU. La razón es práctica. La salida de un bloque pasa por una ReLU, que anula todos los valores negativos, y para una imagen de ruido la mayoría de las posiciones quedan en cero: en la etapa 4, el 56 % de las posiciones centrales. Una unidad en cero no transmite gradiente, de modo que la optimización no puede empezar. Antes de la ReLU las activaciones toman valores de ambos signos y el gradiente existe siempre.

3.3. Preprocesado

Las imágenes se reescalan a 224×224 píxeles y se normalizan por canal de color con la media y la desviación típica de ImageNet, que son las que se usaron al entrenar el modelo. La misma normalización se aplica a las fotografías reales y a las imágenes que se optimizan. Si ambas no coincidieran, las visualizaciones corresponderían a una entrada distinta de la que el modelo vio durante el entrenamiento, sin que se produjera ningún error visible.

4. Método

4.1. El ciclo de optimización

La figura 2 resume una iteración. Unos parámetros generan la imagen a través de una parametrización (sección 4.3); la imagen sufre una transformación aleatoria (sección 4.4), se normaliza y atraviesa la red congelada; de la capa elegida se extrae el escalar h que se quiere maximizar (sección 4.2), y la pérdida combina $-h$ con las penalizaciones (sección 4.5). El gradiente de la pérdida respecto a los parámetros de la imagen se calcula por retropropagación y el optimizador Adam [4] actualiza esos parámetros. El proceso se repite durante un número fijo de pasos, 256 por defecto.

Antes de iniciar la optimización se comprueba que el objetivo transmite gradiente a la imagen. Si el gradiente es exactamente nulo, la unidad está anulada por una ReLU y la optimización devolvería

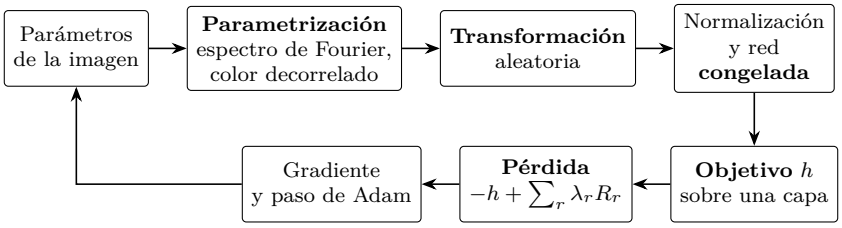


Figura 2: Una iteración del ciclo de optimización. El gradiente atraviesa la red sin modificarla y actualiza únicamente los parámetros de la imagen.

el ruido inicial sin modificarlo, sin que nada lo indicase. En ese caso el proceso se detiene y se informa de ello. Cuando el objetivo combina varias unidades, la comprobación se hace sobre cada una por separado, ya que una unidad activa basta para que el objetivo conjunto tenga gradiente y oculte a otra que no lo tiene.

4.2. Objetivos de optimización

El objetivo h determina qué parte de la red se estudia. Sea $A = A^{(n)}(\mathbf{x})$ la activación de la capa elegida. Se han implementado los cinco objetivos básicos del artículo, representados en la figura 3:

$$h_{\text{neurona}} = A_{z,i,j}, \qquad h_{\text{canal}} = \frac{1}{HW}\sum_{i,j} A_{z,i,j}, \qquad h_{\text{capa}} = \frac{1}{CHW}\sum_{c,i,j} A^2_{c,i,j}, \tag{2}$$

$$h_{\text{logit}} = \ell_k, \qquad h_{\text{prob}} = \text{softmax}(\boldsymbol{\ell})_k = \frac{e^{\ell_k}}{\sum_j e^{\ell_j}}.$$

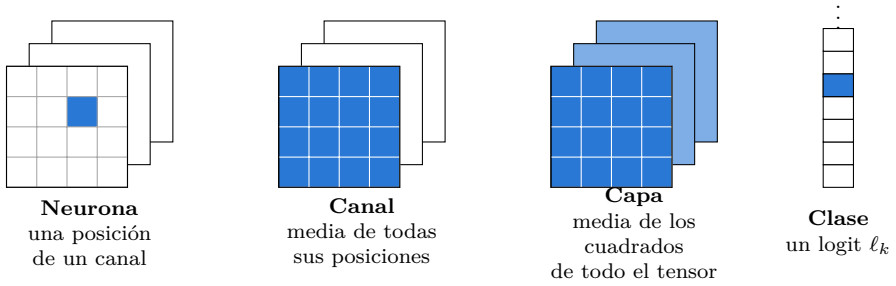


Figura 3: Elementos de la red que intervienen en cada objetivo. Los tres primeros operan sobre el tensor de activaciones de una capa convolucional, representado como una pila de canales; el último, sobre el vector de salida.

El objetivo de *neurona* maximiza una sola posición de un canal, por defecto la central. Solo recibe gradiente de su campo receptivo, de modo que en las capas iniciales afecta a una fracción pequeña de la imagen. El de *canal* promedia todas las posiciones y produce un patrón que se repite por toda la imagen; es el objetivo empleado en la mayoría de las figuras. El de *capa*, conocido como DeepDream, no selecciona ninguna unidad: amplifica cualquier patrón al que la capa ya responda. El cuadrado hace que concentrar la activación en pocos canales puntúe más que repartirla, y en las capas previas a la ReLU recompensa también las activaciones negativas intensas.

Los dos objetivos de clase actúan sobre la salida de la red. El de *logit* maximiza la puntuación de una especie. El de *probabilidad* parece equivalente, pero tiene dos inconvenientes. La probabilidad puede aumentar reduciendo los logits de las demás clases en lugar de aumentar el de la clase elegida. Además, su derivada respecto a su propio logit, $p_k(1 - p_k)$, tiende a cero cuando p_k se acerca a uno: la optimización se detiene en cuanto la red está segura de la clase, aunque la imagen apenas la represente. Por ambas razones las visualizaciones de clase utilizan el logit, y la probabilidad se conserva para mostrar la diferencia.

4.3. Parametrización de la imagen

La forma de representar la imagen como variable de optimización determina qué cambios resultan fáciles para el optimizador, y es el factor que más influye en la calidad del resultado. La representación directa por píxeles favorece el ruido adversarial: un paso de gradiente modifica cada píxel de forma independiente, y los cambios de alta frecuencia son tan fáciles de producir como los de baja.

Siguiendo el artículo, la imagen se parametriza en el dominio de la frecuencia y en un espacio de color decorrelado. Los parámetros θ son los coeficientes de la transformada de Fourier de cada canal de color, y la imagen se obtiene como

$$\mathbf{x}(\theta) = \sigma\Big(M \cdot \tfrac{1}{4}\,\mathcal{F}^{-1}\big(s \odot \theta\big)\Big), \qquad s(u,v) = \frac{\sqrt{HW}}{\text{máx}\big(\sqrt{u^2+v^2},\, 1/\,\text{máx}(H,W)\big)^{d}}, \tag{3}$$

donde (u, v) es la frecuencia espacial de cada coeficiente, $d = 1$ es el exponente de atenuación, $\mathcal{F}^{-1}$ es la transformada inversa, M es una matriz de 3×3 que se aplica al color de cada píxel y σ es la función sigmoide. La figura 4 muestra la secuencia.

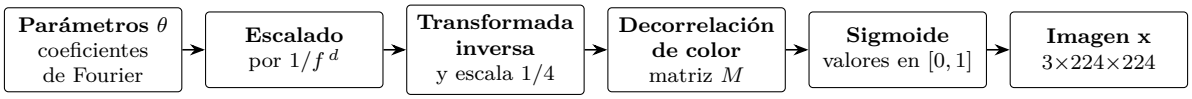


Figura 4: Generación de la imagen a partir de sus parámetros. El optimizador actúa sobre los coeficientes de la izquierda; la imagen de la derecha es la que recibe la red.

Cada pieza cumple una función concreta. El escalado por $1/f^d$ hace que un paso de gradiente de igual tamaño desplace mucho más las frecuencias bajas que las altas; equivale a un precondicionamiento del problema, que reparte el progreso entre escalas en lugar de concentrarlo en el detalle fino. El término $1/\,\text{máx}(H, W)$ evita la división por cero en la componente continua.

El color recibe un tratamiento análogo. En las imágenes naturales los canales R, G y B están fuertemente correlacionados: un píxel claro suele serlo en los tres. Por ello los parámetros no describen cada píxel en RGB, sino en una base de tres componentes incorreladas, obtenida a partir de la covarianza de color de ImageNet: la primera equivale aproximadamente a la luminosidad, y las otras dos, a contrastes entre colores, rojo frente a azul y verde frente a magenta. La matriz M transforma esa base en RGB. Como el escalado por frecuencias, es una forma de precondicionamiento, y evita que la imagen derive hacia un tono uniforme.

La sigmoide mantiene los valores en $[0, 1]$ sin anular el gradiente, a diferencia de un recorte. Los parámetros se inicializan con ruido gaussiano de desviación 0,01.

Para poder comparar, se conserva también la parametrización directa por píxeles, $\mathbf{x} = \sigma(\theta)$, con la misma sigmoide. Así, la única diferencia entre ambas es el precondicionamiento.

4.4. Robustez a transformaciones

Un patrón adversarial depende de su posición exacta en la imagen: si la imagen se desplaza un píxel, deja de funcionar. La robustez a transformaciones aprovecha esta fragilidad. En cada iteración, la imagen se somete a una transformación aleatoria antes de entrar en la red, de modo que el objetivo efectivo pasa a ser el valor esperado $\mathbb{E}_T\big[h(A(T(\mathbf{x})))\big]$ sobre una familia de transformaciones T. Cada iteración usa una única transformación, que constituye una estimación estocástica de esa esperanza. Solo sobreviven los patrones que activan la unidad desde cualquier encuadre.

La transformación encadena seis operaciones (figura 5), con los valores que el artículo utiliza para sus figuras. Cuando se optimizan varias imágenes a la vez, todas reciben la misma transformación.

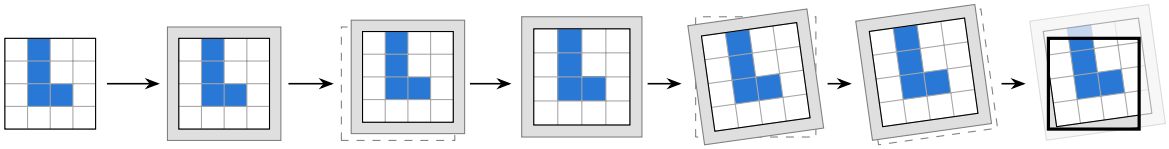


Figura 5: Transformación aleatoria aplicada en cada iteración, ilustrada sobre una imagen esquemática de 4×4 píxeles. Cada paso actúa sobre el resultado del anterior; la línea discontinua marca la posición previa y la zona gris, el relleno. Los desplazamientos, la escala y el ángulo están exagerados para que se aprecien. En cada iteración el factor de escala se elige entre cinco valores, y el ángulo, entre los once enteros del intervalo.

4.5. Penalizaciones de frecuencia

La tercera forma de evitar el ruido es penalizarlo de forma explícita en la pérdida. Se han implementado las tres penalizaciones que el artículo compara, todas promediadas sobre los elementos de la imagen:

$$R_{L1}(\mathbf{x}) = \text{media } |\mathbf{x}|, \tag{4}$$

$$R_{TV}(\mathbf{x}) = \text{media } |x_{i+1,j} - x_{i,j}| + \text{media } |x_{i,j+1} - x_{i,j}|, \tag{5}$$

$$R_{\text{des}}(\mathbf{x}) = \tfrac{1}{2} \text{ media } \big(\mathbf{x} - \text{sg}(B * \mathbf{x})\big)^2. \tag{6}$$

La norma $L1$ empuja la imagen hacia valores pequeños. La variación total R_{TV} mide la diferencia entre píxeles vecinos, que es alta en presencia de ruido de alta frecuencia. La penalización de desenfoque R_{des} compara la imagen con una versión suavizada por un *kernel* B de 3×3; el operador sg detiene el gradiente a través del término suavizado, de modo que minimizarla equivale a desenfocar ligeramente la imagen en cada paso. Con la parametrización de Fourier y la robustez activas, estas penalizaciones rara vez son necesarias, y sus pesos son cero por defecto; se mantienen para poder estudiar su efecto.

4.6. Interacción entre neuronas

Las unidades de una red no actúan aisladas. El artículo propone considerar el *espacio de activaciones* de una capa, en el que cada canal es un eje y cualquier combinación de canales es un vector. Se han implementado tres objetivos que exploran este espacio.

Direcciones. El objetivo de dirección generaliza el de canal a un vector $\mathbf{v} \in \mathbb{R}^C$ arbitrario:

$$h_{\text{dir}} = \frac{1}{HW} \sum_{i,j} \mathbf{v} \cdot A_{:,i,j}. \tag{7}$$

Si $\mathbf{v}$ es el vector unitario del canal z, se recupera el objetivo de canal. Con $\mathbf{v}$ aleatorio, de norma uno, se visualiza una dirección que no corresponde a ninguna unidad individual. Cada dirección se identifica por la semilla con la que se generó, lo que la hace reproducible.

Suma de neuronas. Maximizar la suma de dos objetivos de canal produce una imagen que debe satisfacer a ambos a la vez, y muestra cómo se combinan sus patrones.

Interpolación. Una interpolación optimiza simultáneamente una fila de N imágenes. La imagen m, con $m = 0, \ldots, N-1$, maximiza una combinación lineal de dos canales a y b de la misma capa,

$$h_m = (1 - w_m)\, h_{\text{canal}}(a) + w_m\, h_{\text{canal}}(b), \qquad w_m = \frac{m}{N-1}, \tag{8}$$

de modo que la primera imagen corresponde solo al canal a, la última solo al b y las intermedias a mezclas progresivas. Si cada imagen se optimiza por separado, cada una produce una composición distinta y la fila no se percibe como una transición. Para evitarlo se añaden dos mecanismos, representados en la figura 6.

El primero es una penalización de *alineación* sobre las activaciones de capas inferiores, que hace que imágenes próximas en la fila activen esas capas de forma parecida:

$$R_{\text{alin}} = \sum_{l} \sum_{d=1}^{4} \sum_{m} \frac{1}{5^d} \operatorname{media}\big(A_m^{(l)} - A_{m+d}^{(l)}\big)^2, \tag{9}$$

donde l recorre las etapas 2 y 3 de la red y el factor $1/5^d$ reduce el peso de las parejas más alejadas.

El segundo es una *parametrización compartida*. Además de sus parámetros propios de Fourier, cada imagen suma seis tensores de baja resolución, comunes a toda la fila, de tamaños entre 1/2 y 1/16 del original. Cuatro de ellos son idénticos para todas las imágenes; los otros dos tienen dos fotogramas clave, que se interpolan linealmente a lo largo de la fila. Así, la estructura de baja frecuencia se comparte y cambia de forma gradual, mientras que el detalle de cada imagen queda libre.

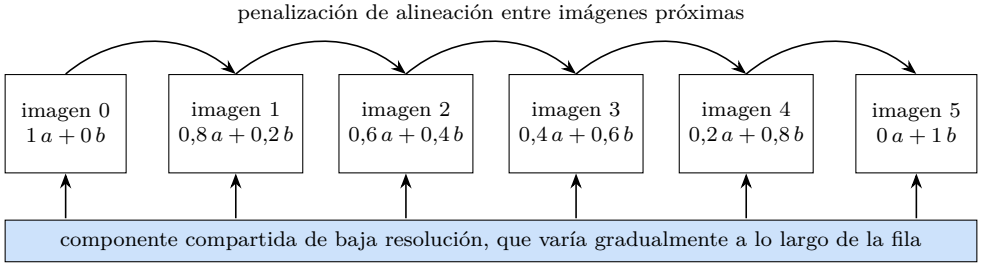


Figura 6: Interpolación entre los canales a y b con $N = 6$ imágenes. Cada imagen recibe su propio peso. Las flechas superiores representan la alineación entre imágenes vecinas, que en la implementación se extiende hasta distancia 4; la banda inferior, la parte de la parametrización común a toda la fila.

El peso de la alineación se fijó en 100 mediante un barrido sobre un par de canales de la etapa 4. Con pesos de 0 y 10 la fila salta de una composición a otra en la mitad; con 100 la transición es gradual y los extremos conservan en torno al 90 % de su activación; con 1000 las imágenes apenas difieren entre sí y pierden más de la mitad de la activación.

4.7. Diversidad

Una unidad puede responder a varios estímulos distintos, y una sola visualización muestra solo uno de ellos. Para obtener varios, se optimizan N imágenes del mismo objetivo y se añade un término que las obliga a ser distintas entre sí. El artículo toma la idea de la transferencia de estilo. Para cada imagen m se calcula la matriz de Gram de los canales de una capa,

$$G_m = \tilde{A}_m \tilde{A}_m^{\top} \in \mathbb{R}^{C \times C}, \qquad \tilde{A}_m \in \mathbb{R}^{C \times HW}, \tag{10}$$

donde $\tilde{A}_m$ es la activación con las posiciones espaciales aplanadas. El elemento $G_m[c, c']$ mide en qué medida los canales c y c' se activan juntos, con independencia de dónde lo hagan; describe el *estilo* de la imagen y no su composición. La penalización es la similitud coseno media entre las matrices de todas las parejas,

$$R_{\text{div}} = \frac{1}{N} \sum_{a \neq b} \frac{\langle G_a, G_b \rangle}{\|G_a\| \, \|G_b\|}, \tag{11}$$

que vale 0 cuando ninguna pareja comparte estilo y $N - 1$ cuando todas las imágenes son iguales. La figura 7 resume el cálculo.

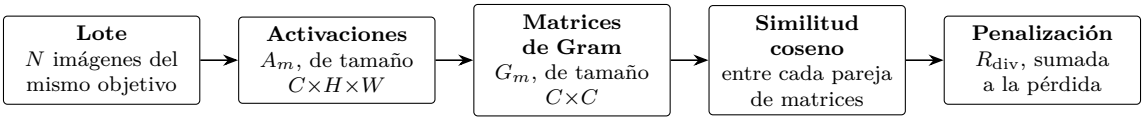


Figura 7: Cálculo del término de diversidad. Al minimizar la similitud entre matrices de Gram, cada imagen del lote activa la capa con un estilo distinto.

Como la alineación y la parametrización compartida acercan las imágenes del lote entre sí, ambas se desactivan cuando se aplica la diversidad. Los objetivos de clase actúan sobre una salida sin dimensión espacial, que no admite matriz de Gram; en ese caso la diversidad se mide en una capa convolucional, la etapa 4 en nuestros experimentos. El peso se fijó en 10 mediante un barrido con lotes de cuatro imágenes de un canal de la etapa 4. Sin el término, las cuatro imágenes son prácticamente iguales, con una similitud media de 0,96, aunque partan de ruidos distintos. Con peso 10 la similitud baja a 0,33 y cada imagen muestra un patrón diferente, conservando el 79 % de la activación media. Con 100, tres de las cuatro imágenes pierden casi toda la activación.

5. Herramienta de inspección

Sobre estas técnicas se ha construido una herramienta, accesible desde una interfaz web y desde la línea de órdenes, que aplica el recorrido de la figura 8 a una fotografía elegida por el usuario.

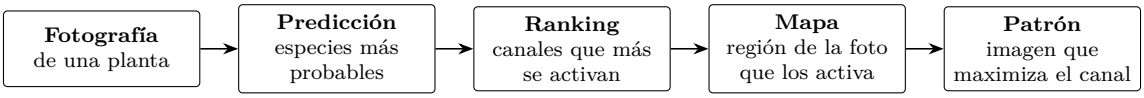


Figura 8: Recorrido de la herramienta sobre una fotografía. Las tres primeras etapas requieren una sola pasada por la red; la última es una optimización completa.

La *predicción* es la salida softmax de la red. El *ranking* calcula, para cada canal de la capa elegida, su activación media sobre la fotografía, que es el objetivo de canal evaluado en lugar de maximizado, y los ordena de mayor a menor. Las escalas de activación difieren mucho de una capa a otra, por lo que el orden solo es comparable dentro de una misma capa.

El *mapa de activación* recupera la información espacial que el ranking descarta. Toma la rejilla de activaciones del canal antes del promedio (14×14 en la etapa 3, 7×7 en la etapa 4), anula los valores negativos, que indican que la región inhibe el canal, la normaliza por su máximo y la reescala por interpolación bilineal al tamaño de la fotografía, sobre la que se superpone. El mapa relaciona así un valor del ranking con una zona concreta de la imagen.

El *patrón* es la visualización del mismo canal, obtenida con el ciclo de la sección 4. Como contrapunto, el sistema permite buscar, entre un conjunto de fotografías reales, las que más activan el canal. Comparar el patrón optimizado con esos ejemplos permite comprobar si lo que muestra la visualización aparece también en imágenes reales o es un producto de la optimización.

6. Métricas

Activación. Es el valor del objetivo h. Se registra en cada paso de la optimización, sobre la imagen transformada aleatoriamente, por lo que la curva es ruidosa; el valor que se da como final es el del último paso. Con la robustez activa esa medida se toma, por tanto, sobre una vista transformada: evaluar en su lugar la imagen final sin transformar cambia el resultado en menos del 1 %, precisamente porque el patrón está optimizado para resistir esas transformaciones. Su escala depende de la capa y del objetivo, de modo que solo es comparable entre ejecuciones con el mismo objetivo.

Aspereza. Es la variación total de la imagen final, con la misma expresión (5) que la penalización R_{TV}: la diferencia absoluta media entre píxeles vecinos, en horizontal y en vertical. Cuantifica el contenido de alta frecuencia. Un valor alto indica una imagen dominada por ruido, y uno bajo, una imagen formada por estructuras amplias. Permite comparar configuraciones sin recurrir a la inspección visual.

Similitud de Gram. En los experimentos de diversidad se mide la similitud coseno media entre las matrices de Gram de las imágenes del lote, $R_{\text{div}}/(N-1)$. Vale 1 cuando todas comparten estilo y 0 cuando no comparten nada.

Diferencia entre vecinas. En las interpolaciones se mide la diferencia absoluta media, píxel a píxel, entre imágenes consecutivas de la fila. Valores bajos y uniformes indican una transición gradual.

Acierto top-1 y top-5. Fracción de fotografías cuya especie correcta es la más probable, o está entre las cinco más probables, según la red. No mide la visualización, sino que verifica que el modelo y el preprocesado son los correctos (sección 8.1).

7. Configuración experimental

La tabla 1 recoge los valores empleados en todos los experimentos salvo que se indique otra cosa. Todos los hiperparámetros se definen en un único fichero de configuración, y la semilla aleatoria se fija en un único punto del código.

Tabla 1: Configuración empleada en los experimentos.

Parámetro	**Valor**
Modelo	ResNet-18 preentrenado en Pl@ntNet-300K, 1081 clases
Tamaño de la imagen	224×224 píxeles
Parametrización	Fourier con color decorrelado, $d = 1$, inicialización $\mathcal{N}(0,\ 0{,}01^2)$
Optimizador	Adam, tasa de aprendizaje 0,05, 256 pasos
Robustez	relleno 16, traslaciones de ± 16 y ± 8, escala 0,95–1,05, rotación $\pm 5°$
Penalizaciones de frecuencia	$\lambda_{L1} = \lambda_{TV} = \lambda_{\text{des}} = 0$
Interpolación	6 imágenes por fila; alineación en las etapas 2 y 3, peso 100
Diversidad	4 imágenes por lote, peso 10
Normalización	media y desviación típica de ImageNet
Semilla	42

El artículo emplea 2560 pasos por imagen. Con 256 pasos una visualización tarda unos 18 segundos en CPU y unos 3 en una GPU de gama media, frente a unos tres minutos en CPU con 2560; las imágenes resultan algo menos refinadas. Cada ejecución registra en Weights & Biases la activación, la pérdida y la aspereza en cada paso, y la imagen cada 32 pasos. Entre ejecuciones en CPU, la activación final puede variar en torno a un 1 % por el no determinismo de algunas operaciones.

8. Resultados

8.1. Verificación del modelo

Un error en la normalización de entrada o en la correspondencia entre clases y especies no produce ningún fallo visible: las visualizaciones se siguen generando, pero corresponden a otra unidad o a un modelo mal cargado. Antes de interpretar el modelo se comprobó, por tanto, que clasifica correctamente. Sobre 120 fotografías del conjunto de prueba, una por especie, tomadas entre las 144 especies con al menos 300 imágenes de entrenamiento, la red acierta el 80,0 % en top-1 y el 95,0 % en top-5.

La misma medida resolvió una cuestión que los pesos publicados no documentan: con qué estadísticas se normalizó la entrada durante el entrenamiento. Con las de ImageNet el acierto top-1 es del 80,0 %, y con las de Pl@ntNet-300K, del 79,2 %, con el mismo top-5. La diferencia es de una imagen sobre 120, por lo que la elección no afecta a los resultados.

8.2. Recorrido completo

La figura 9 muestra el resultado de la herramienta sobre una fotografía del conjunto de prueba.



Figura 9: Recorrido completo sobre una misma fotografía. Izquierda: la entrada, clasificada como *Lactuca serriola* con una probabilidad del 63,0 %. Centro: mapa de activación del canal 362 de la última convolución de la etapa 4, el más activo de esa capa para esta fotografía. Derecha: imagen optimizada desde ruido para maximizar ese mismo canal.

8.3. Efecto de la robustez a transformaciones

El artículo sostiene que, sin regularización, la optimización alcanza activaciones más altas con imágenes peores. Las dos métricas de la sección 6 permiten comprobarlo de forma cuantitativa. La tabla 2 compara la optimización de un mismo canal con y sin robustez a transformaciones.

Tabla 2: Ablación de la robustez a transformaciones. Canal 181 de la última convolución de la etapa 3, 256 pasos.

	Activación final	**Aspereza**
Con robustez	+4,93	0,164
Sin robustez	+7,15	0,245
Variación	+45 %	+49 %

Sin robustez, la activación aumenta un 45 % y la aspereza un 49 %. El optimizador obtiene un valor mejor del objetivo a costa de una imagen con más contenido de alta frecuencia, que es el

comportamiento descrito en la sección 2. Desactivar la parametrización de Fourier también aumenta la aspereza de la imagen final.

9. Adaptaciones respecto al artículo

El artículo aplica sus técnicas a una red GoogLeNet entrenada en ImageNet; este trabajo las aplica a redes ResNet entrenadas en Pl@ntNet-300K. El número de pasos se ha reducido de 2560 a 256 por coste, pero puede restablecerse desde la configuración. Los pesos de las penalizaciones de las figuras del artículo no son trasladables, porque allí las penalizaciones se suman sobre todos los elementos de la imagen y aquí se promedian. Por la misma razón, los pesos de la alineación y de la diversidad se han ajustado con barridos propios. Por último, el artículo describe una tercera familia de regularización, los *priors* aprendidos mediante un modelo generativo, que queda fuera del alcance de este trabajo porque requiere entrenar dicho modelo.

10. Conclusiones

Se ha aplicado a redes ResNet entrenadas sobre Pl@ntNet-300K el conjunto de técnicas de visualización de características descrito por Olah et al.: los objetivos sobre neuronas, canales, capas y clases; la parametrización en el dominio de la frecuencia con color decorrelado; la robustez a transformaciones; las penalizaciones de frecuencia; la interacción entre neuronas y la diversidad. Sobre ellas se ha construido una herramienta que relaciona una fotografía con las unidades que activa, con la región donde lo hace y con el patrón que cada unidad detecta. La ablación de la robustez confirma de forma cuantitativa que la regularización sacrifica valor del objetivo a cambio de imágenes interpretables, que es el argumento central del artículo.

Referencias

[1] C. Olah, A. Mordvintsev y L. Schubert. *Feature Visualization*. Distill, 2017. DOI: 10.23915/distill.00007.

[2] C. Garcin, A. Joly, P. Bonnet et al. *Pl@ntNet-300K: a plant image dataset with high label ambiguity and a long-tailed distribution*. NeurIPS Datasets and Benchmarks, 2021.

[3] K. He, X. Zhang, S. Ren y J. Sun. *Deep Residual Learning for Image Recognition*. CVPR, 2016.

[4] D. P. Kingma y J. Ba. *Adam: A Method for Stochastic Optimization*. ICLR, 2015.